

СИСТЕМА ЭЛЕКТРОННОЙ ПОДАЧИ ЗАЯВОК «БЮРО
ПРОПУСКОВ»

ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ

Развёртывание, сопровождение и техническое описание системы

Аудитория: ИТ-специалисты, администраторы, инженеры сопровождения

Версия документа: 1.0

Дата: 20.06.2026

Документ актуален на 20.06.2026, соответствует текущей версии системы (ветка dev).

Оглавление

1. Введение и назначение	4
1.1. Назначение документа	4
1.2. Назначение системы	4
1.3. Состав документа	4
1.4. Термины и сокращения	4
2. Архитектура и технологический стек	6
2.1. Технологический стек	6
2.2. Слои и взаимодействие	6
2.3. Масштаб проекта	7
3. Требования к окружению	8
3.1. Развёртывание через Docker (рекомендуемое)	8
3.2. Развёртывание без Docker	8
3.3. Требования к production-серверу	8
3.4. Порты сервисов	8
4. Развёртывание	9
4.1. Развёртывание через Docker Compose	9
4.2. Развёртывание без Docker	10
4.3. Production-развёртывание	10
5. Конфигурация (переменные окружения)	12
5.1. База данных и сервер	12
5.2. Аутентификация (JWT)	12
5.3. Шифрование ПДн (152-ФЗ)	12
5.4. CORS и загрузка файлов	13
5.5. Rate limiting и пагинация	13
5.6. Прочие настройки	13
6. База данных	15
6.1. Миграции (AutoMigrate)	15
6.2. Ключевые доменные модели	15
6.3. Группы таблиц	16
7. Программный интерфейс (API)	17
7.1. Документация API (Swagger)	17
7.2. Конвейер обработки запроса	17
7.3. Группы маршрутов	17
8. Безопасность	19
8.1. Аутентификация	19

8.2. Авторизация (RBAC).....	19
8.3. Защита от типовых атак.....	19
8.4. Сеть и инфраструктура.....	20
9. Соответствие 152-ФЗ и защита ПДн.....	21
9.1. Шифрование персональных данных.....	21
9.2. Аудит доступа к ПДн.....	21
9.3. Журнал согласий.....	21
10. CI/CD.....	22
10.1. Рабочие процессы (workflows).....	22
10.2. Сканирование безопасности.....	22
11. Сопровождение и эксплуатация.....	24
11.1. Резервное копирование PostgreSQL.....	24
11.2. Мониторинг и проверка состояния.....	24
11.3. Обновление.....	24
11.4. Откат.....	25
12. Архитектурные решения (ADR).....	26
Лист регистрации изменений.....	27

1. Введение и назначение

1.1. Назначение документа

Настоящий документ — техническая документация системы электронной подачи заявок «Бюро пропусков». Он описывает архитектуру системы, требования к окружению, порядок развёртывания, конфигурацию, устройство базы данных, программный интерфейс (API), реализованные меры безопасности и соответствие 152-ФЗ, организацию CI/CD и регламент сопровождения. Документ рассчитан на ИТ-специалистов: системных администраторов, инженеров развёртывания и сопровождения, разработчиков.

1.2. Назначение системы

Система автоматизирует работу бюро пропусков: электронную подачу, согласование и учёт заявок на проход людей и проезд транспорта на территорию. Пользователи оформляют заявки и ведут справочники сотрудников и автомобилей, ответственные согласуют заявки, администраторы управляют пользователями, организациями, правами доступа и системными справочниками. Система реализована как веб-приложение и не требует установки клиентского программного обеспечения, кроме современного браузера.

Ключевые подсистемы:

- **Заявки** — оформление, согласование, контроль статусов, история изменений, типы вложений (автозаявки, проведение работ, разовые пропуска).
- **Справочники** — сотрудники, автомобили, организации, компании, отделы, места выгрузки, форматы номеров, гражданство.
- **Доступ** — аутентификация, ролевая модель прав (RBAC), журналы входов и отказов.
- **Защита ПДн** — шифрование персональных данных, аудит доступа, журнал согласий (152-ФЗ).
- **Администрирование** — управление пользователями, системные таблицы, новости, объявления, документы, обратная связь.

1.3. Состав документа

Документ построен по логике жизненного цикла развёртывания и эксплуатации: от обзора архитектуры и требований к окружению — через установку и настройку — к описанию базы данных, API, безопасности и регламентов сопровождения. Раздел «Архитектурные решения» фиксирует обоснование ключевого технологического выбора.

1.4. Термины и сокращения

Термин	Расшифровка
SPA	Single Page Application — одностраничное веб-приложение, работающее в браузере
API	Application Programming Interface — программный интерфейс взаимодействия с сервером
REST	Архитектурный стиль построения HTTP-API

JWT	JSON Web Token — подписанный токен для аутентификации
ORM	Object-Relational Mapping — отображение объектов на таблицы БД (здесь — GORM)
RBAC	Role-Based Access Control — управление доступом на основе ролей и прав
ПДн	Персональные данные
152-ФЗ	Федеральный закон «О персональных данных»
ADR	Architecture Decision Record — запись архитектурного решения
CI/CD	Continuous Integration / Continuous Delivery — непрерывная интеграция и доставка
T/C	Транспортное средство

2. Архитектура и технологический стек

Система построена по классической трёхзвенной схеме: клиентский слой (SPA на Vue 3), серверный слой (REST API на Go и фреймворке Echo) и слой хранения данных (PostgreSQL). Все компоненты разворачиваются в контейнерах Docker и работают в изолированной сети за обратным прокси nginx.

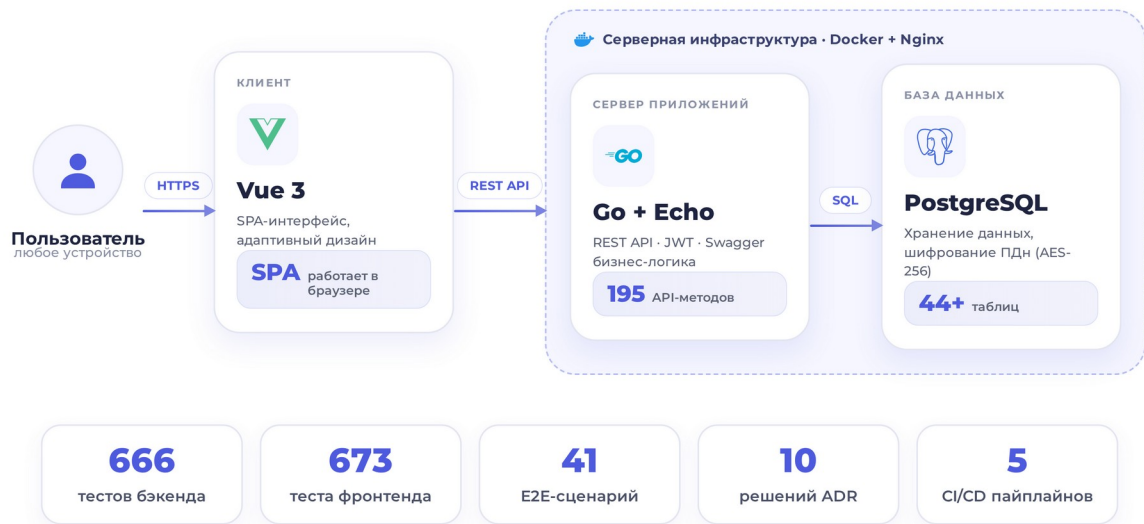


Рисунок 1 — Общая архитектура системы и масштаб проекта

2.1. Технологический стек

Слой	Технологии	Назначение
Клиент	Vue 3, Vue Router 4, Pinia, Vite	SPA-интерфейс, маршрутизация, управление состоянием, сборка
Сервер приложений	Go 1.25, Echo v4, GORM	REST API, бизнес-логика, ORM, middleware
База данных	PostgreSQL 16	Хранение данных, шифрование ПДн на уровне полей
Инфраструктура	Docker Compose, nginx, GitHub Actions	Контейнеризация, обратный прокси, CI/CD

2.2. Слои и взаимодействие

- Клиентский слой** — браузер пользователя загружает SPA (Vue 3) и обменивается с сервером по REST API поверх HTTPS. Состояние токенов, прав и интерфейса хранится в Pinia-сторах.

- **Серверный слой** — Go-сервис на Echo принимает HTTP-запросы, проводит их через конвейер middleware (идентификатор запроса, логирование, восстановление после паник, CORS, rate limiting, аудит ПДн, проверка JWT), вызывает сервисы бизнес-логики и обращается к БД через GORM.
- **Слой данных** — PostgreSQL хранит все сущности. Схема поддерживается GORM AutoMigrate. Персональные данные шифруются на уровне полей через GORM-хуки.

2.3. Масштаб проекта

Объём реализации в цифрах (по ветке dev):

Показатель	Значение
API: путей / операций / моделей	195 / 249 / 208
Таблиц базы данных (AutoMigrate)	44+
Доменных моделей	25+
Vue-компонентов / view	142 / 28
Тесты backend (Go)	666 (с race-детектором в CI)
Тесты frontend (vitest)	673
Сценарии E2E (Playwright)	41 (матрица из 4 шардов)
Архитектурные решения (ADR)	10
Рабочие процессы CI/CD	5

Примечание: Метрика покрытия в процентах в репозитории не ведётся. Качество тестирования характеризуется числом тестов и их уровнями: backend-тесты интеграционные (поднимаются против реального PostgreSQL), E2E — сквозные на живом сервере.

3. Требования к окружению

3.1. Развёртывание через Docker (рекомендуемое)

Рекомендуемый способ — контейнерное развёртывание. Требования к хосту:

- **Docker** — версии 24 и выше.
- **Docker Compose** — версии v2.
- **Оперативная память** — минимум 2 ГБ, рекомендуется 4 ГБ.
- **Операционная система** — любой Linux-дистрибутив с поддержкой Docker (production-развёртывание выполняется на Linux-VPS).

3.2. Развёртывание без Docker

При установке напрямую на хост требуются:

- **Go** — версии 1.25 и выше (для сборки и запуска backend).
- **Node.js** — версии 18 и выше, а также npm (для сборки frontend).
- **PostgreSQL** — версии 16.

3.3. Требования к production-серверу

- **CPU** — 2 и более ядер.
- **RAM** — 4 ГБ и более.
- **ПО** — Docker и Docker Compose.
- **Сеть** — домен с настроенной DNS А-записью (для выпуска TLS-сертификата).

3.4. Порты сервисов

Состав сервисов и используемые порты в развёртывании по умолчанию:

Сервис	Порт	Назначение
db	5432	PostgreSQL 16 — база данных
go-backend	8080	Go API (REST)
frontend	8081	Vue 3 (dev-сервер) / собранный SPA
pgadmin	8082	Веб-интерфейс администрирования БД (только dev/staging)

Важно: Сервис pgadmin доступен только в dev/staging-конфигурации. В production-окружении он отсутствует: прямой web-доступ к БД на production недопустим.

4. Развёртывание

4.1. Развёртывание через Docker Compose

Способ по умолчанию для dev и базового запуска. Схема контейнеров и сети показана на рисунке 2.

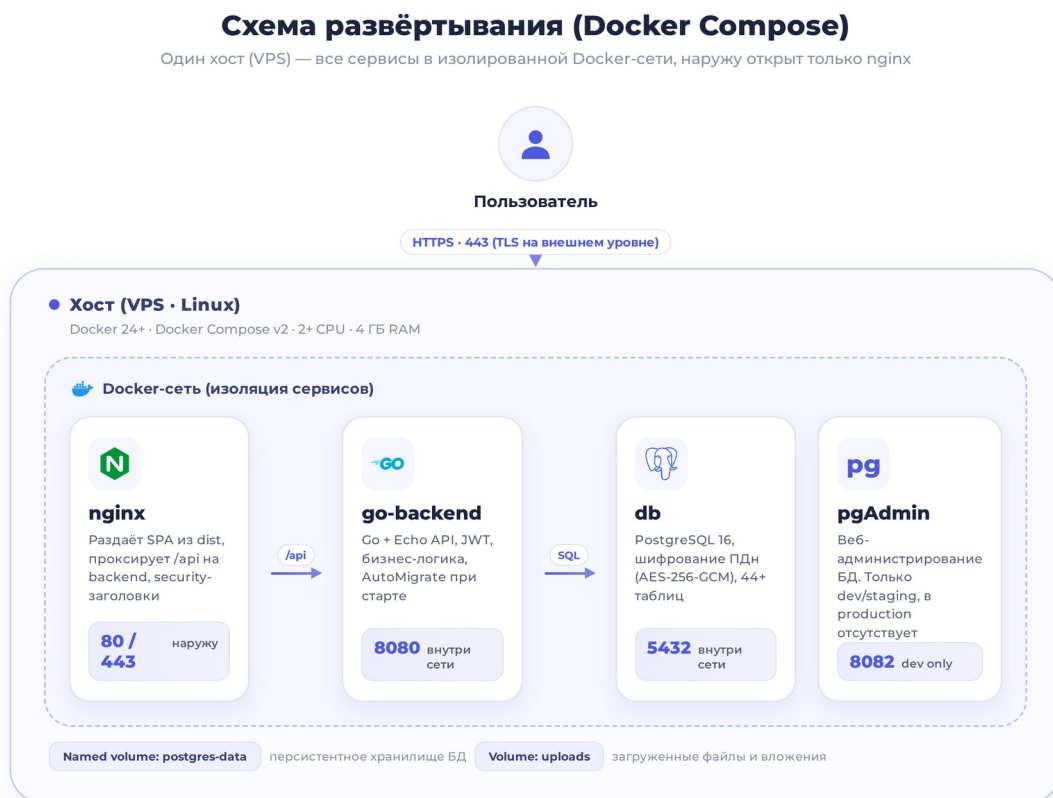


Рисунок 2 — Схема развёртывания (Docker Compose)

Шаг 1. Клонировать репозиторий

```
git clone <repo> && cd systemburo
```

Шаг 2. Создать файл окружения

Скопируйте шаблон и задайте безопасные значения секретов (см. раздел 5).

```
cp .env.example .env
# задать DB_PASSWORD, JWT_SECRET, JWT_REFRESH_SECRET,
# DATA_ENCRYPTION_KEY (openssl rand -hex 32)
```

Шаг 3. Запустить сервисы

```
make up
# или: docker compose up -d
```

Шаг 4. Проверить работоспособность

```
curl http://localhost:8080/health
# {"status":"ok"}
```

Шаг 5. Создать тестового администратора (dev)

Команда seed создаёт учётную запись для входа.

```
make seed # buropropuskov / admin123
# свой пароль: make seed PASS=mypassword
# демо-набор данных: make seed-demo
```

Точки доступа после запуска: Backend — <http://localhost:8080/health>; Frontend — <http://localhost:8081>; pgAdmin — <http://localhost:8082>; Swagger UI — <http://localhost:8080/swagger/index.html> (при включённом SWAGGER_ENABLED).

4.2. Развёртывание без Docker

Раздельная установка backend и frontend на хост.

Backend:

```
go mod download
go install github.com/swaggo/swag/cmd/swag@latest
swag init -g cmd/server/main.go -o docs

export DATABASE_URL=postgres://user:pass@localhost:5432/auto_registry
export JWT_SECRET=your-secret-min-32-chars-here!!!!
export JWT_REFRESH_SECRET=your-refresh-secret-min-32-chars!
export BIND_PORT=8080

go run ./cmd/server
```

Frontend:

```
cd frontend
npm ci
npm run serve # dev-сервер на :8081
```

База данных:

```
createdb auto_registry
# миграции выполняются автоматически при старте backend (GORM AutoMigrate)
```

4.3. Production-развёртывание

Развёртывание на VPS с доменом и TLS.

Шаг 1. Подготовить каталог и окружение

```
git clone <repo> /opt/systemburo
cd /opt/systemburo
```

```
cp .env.example .env
# заполнить .env production-значениями
```

Шаг 2. Собрать и запустить

```
make deploy-build
make deploy-up
```

Шаг 3. Настроить SSL (certbot)

```
apt install certbot python3-certbot-nginx
certbot --nginx -d your-domain.com
```

Шаг 4. Включить HTTPS-редирект и HSTS

После выпуска сертификата раскомментируйте редирект и добавьте HSTS-заголовок в nginx-конфигурации.

```
# редирект на HTTPS:
return 301 https://$host$request_uri;
# HSTS:
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

Порядок инициализации backend при старте: Загрузка конфигурации из env -> настройка логирования -> инициализация шифрования -> подключение к PostgreSQL -> AutoMigrate (44+ таблиц, идиempотентно) -> seed справочников по умолчанию -> создание Echo и middleware -> регистрация маршрутов -> (опц.) Swagger UI -> graceful shutdown по SIGINT/SIGTERM.

5. Конфигурация (переменные окружения)

Все настройки задаются переменными окружения (загружаются из `.env`). Значения валидируются при старте: при ошибке конфигурации сервис не запускается. Ниже — полный перечень переменных.

5.1. База данных и сервер

Переменная	Назначение	Обяз.	По умолчанию
DATABASE_URL	Строка подключения к PostgreSQL (postgres://...)	Да	—
BIND_HOST	Адрес прослушивания backend	Нет	0.0.0.0
BIND_PORT	Порт backend (в <code>.env.example</code> и <code>compose</code> — 8080)	Нет	8090
LOG_LEVEL	Уровень логирования: debug/info/warn/error	Нет	info
SWAGGER_ENABLE	Включение Swagger UI (не для production)	Нет	false

5.2. Аутентификация (JWT)

Переменная	Назначение	Обяз.	По умолчанию
JWT_SECRET	Секрет подписи access-токена (≥ 32 символов)	Да	—
JWT_REFRESH_SECRET	Секрет подписи refresh-токена (≥ 32 символов)	Да	—
JWT_ACCESS_TTL	Время жизни access-токена	Нет	15m
JWT_REFRESH_TTL	Время жизни refresh-токена	Нет	168h
COOKIE_SECURE	Флаг Secure на refresh-cookie (false для localhost)	Нет	true

5.3. Шифрование ПДн (152-ФЗ)

Переменная	Назначение	Обяз.	По умолчанию
DATA_ENCRYPTION_KEY	Ключ AES-256 (32 байта hex = 64 символа)	Нет*	"" (выкл.)

REQUIRE_ENCRYPTION	Требовать наличие ключа при старте	Нет	false
--------------------	------------------------------------	-----	-------

Обязательность ключа в production: Формально DATA_ENCRYPTION_KEY необязателен (без него система работает в режиме passthrough — ПДн пишутся в открытом виде). Для соответствия 152-ФЗ ключ ОБЯЗАН быть задан в production. Сгенерировать: openssl rand -hex 32. Установка REQUIRE_ENCRYPTION=true заставляет систему отказывать в старте без ключа.

5.4. CORS и загрузка файлов

Переменная	Назначение	Обяз.	По умолчанию
CORS_ALLOWED_ORIGINS	Список разрешённых источников (через запятую)	Нет	http://localhost:8081
UPLOAD_MAX_FILE_SIZE	Максимальный размер файла, байт	Нет	10485760 (10 МБ)
UPLOAD_ALLOWED_IMAGE_TYPES	Разрешённые MIME-типы изображений	Нет	jpeg, png, webp
UPLOAD_ALLOWED_DOC_TYPES	Разрешённые MIME-типы документов	Нет	pdf, docx
UPLOAD_PATH	Каталог хранения загрузок	Нет	./uploads

5.5. Rate limiting и пагинация

Переменная	Назначение	Обяз.	По умолчанию
RATE_LIMIT_PER_MINUTE	Лимит запросов в окне	Нет	200
RATE_LIMIT_WINDOW_SEC	Длительность окна, сек	Нет	60
LOGIN_RATE_LIMIT_MAX	Лимит попыток /login per-IP	Нет	10
LOGIN_RATE_LIMIT_WINDOW_SEC	Окно для login-лимита, сек	Нет	300
PAGINATION_MAX_LIMIT	Максимальный размер страницы выдачи	Нет	100

5.6. Прочие настройки

Переменная	Назначение	Обяз.	По умолчанию
RESET_TIMEZONE	Часовой пояс	Нет	Europe/Moscow

	ежедневного сброса территориальных статусов		
TELEGRAM_BOT_TOKEN	Токен бота для bug-репортов (опционально)	Нет	""
TELEGRAM_CHAT_ID	Идентификатор чата для bug-репортов	Нет	""

Валидация конфигурации: При старте проверяются: длина JWT-секретов (≥ 32), формат DATABASE_URL (postgres://), допустимость LOG_LEVEL, положительность лимитов загрузки/пагинации/rate-limit, корректность ключа шифрования (если задан) и условие JWT_REFRESH_TTL > JWT_ACCESS_TTL.

6. База данных

Хранилище — PostgreSQL 16. Схема описывается GORM-моделями (struct-теги), миграция выполняется автоматически при старте приложения механизмом AutoMigrate.

6.1. Миграции (AutoMigrate)

- **Идемпотентность** — AutoMigrate безопасно запускать повторно; создаются недостающие таблицы и колонки (44+ таблиц).
- **Совместимость** — миграция детектирует уже существующие таблицы (наследие предыдущего backend) и не ломает их.
- **Seed** — при старте создаются справочники по умолчанию: 6 типов пользователей и набор прав по умолчанию.

Ограничение AutoMigrate: AutoMigrate не удаляет и не переименовывает колонки, не меняет типы и не откатывает изменения, не версионизирует миграции. Для production рекомендуется перейти на файловые миграции (goose / golang-migrate). См. ADR-003.

6.2. Ключевые доменные модели

Основные сущности и их связи:

Модель	Связи	GORM-хуки
User	belongs-to Organization, Company, UserType	—
Application	has-many History, ResponsibleUsers, Approvers, Viewers, Reads	—
Employee	belongs-to Attachment, Citizenship	BeforeSave: шифрование + HMAC; AfterFind: расшифровка
UniqueEmployee	belongs-to Organization, Company	BeforeSave/AfterFind: шифрование
Car	belongs-to Attachment; has-many CarHistory, CarUnloadPlaces	—
Attachment	has-many Cars, Employees, Items	—
SystemTable	has-many Photos, TimeSlots, Fields	—
PDConsent	belongs-to User	—
PDAuditLog	—	Асинхронная запись через middleware

6.3. Группы таблиц

- **Пользователи и доступ** — users, user_types, refresh_tokens, auth_events, permissions, permission_groups, user_groups, user_permission_overrides, role_default_groups, access_denials.
- **Заявки** — applications и связанные: история, ответственные, согласующие, просматривающие, отметки о прочтении, сотрудники и транспорт заявки.
- **Справочники** — organizations, companies, departments, unload_places, citizenships, форматы номеров, system_tables и связанные.
- **Персональные данные** — pd_consent (согласия), pd_audit_logs (аудит доступа к ПДн).

О диаграмме «сущность-связь»: Отдельной ER-диаграммы (визуальной схемы) в репозитории нет — структура описана текстом в моделях и в документации backend. Для визуального исследования связей используется pgAdmin (см. раздел 3.4).

7. Программный интерфейс (API)

Сервер предоставляет REST API. Все ответы используют единый конверт: success, data, meta, error. Защищённые маршруты требуют access-токен в заголовке Authorization: Bearer.

7.1. Документация API (Swagger)

- **Спецификация** — OpenAPI генерируется через swaggo (docs/swagger.json, docs/swagger.yaml, docs/docs.go).
- **Объём** — 195 путей, 249 операций, 208 моделей. Все операции имеют summary и теги (29 тег-групп).
- **Живой интерфейс** — Swagger UI доступен по адресу /swagger/index.html при включённом SWAGGER_ENABLED (в production выключается).

7.2. Конвейер обработки запроса

Каждый запрос проходит цепочку middleware (для защищённых маршрутов добавляется проверка JWT):

```
RequestID -> Logger -> Recover -> CORS -> RateLimit -> PDAudit -> [JWTAuth]
```

Middleware	Назначение
RequestID	Присваивает X-Request-ID для сквозной трассировки
Logger	Структурное логирование запросов
Recover	Перехват паник, возврат корректной ошибки
CORS	Контроль источников, поддержка credentials
RateLimit	Ограничение частоты по IP/токену, очистка раз в 60 с
PDAudit	Асинхронный аудит доступа к ПДн (/employees, /unique-employees, /attachments)
JWTAuth	Проверка Bearer-токена, помещение данных пользователя в контекст
RequirePermission	Проверка прав на отдельных маршрутах (опционально)

7.3. Группы маршрутов

- **Публичные** — /register, /login, /refresh-token, /user-types, /health.
- **Защищённые (JWT)** — 22 группы: applications, cars, organizations, companies, users, employees, system-tables, unload-places, unique-cars, unique-employees, attachments, feedback, permissions, consents, settings и другие.

Полный список: Исчерпывающий перечень эндпоинтов с параметрами и схемами доступен в Swagger UI (/swagger/index.html).

8. Безопасность

8.1. Аутентификация

- **Access-токен** — JWT HS256, stateless, время жизни 15 минут. Алгоритм подписи проверяется (защита от alg-confusion).
- **Refresh-токен** — отдельный JWT, время жизни 168 часов (7 дней). В БД хранится только sha256-хеш, не сам токен.
- **Ротация и обнаружение повторного использования** — токены связаны в семье (family_id); повторное предъявление отозванного токена инвалидирует всю семью (паттерн OWASP/Auth0), grace-window 10 с.
- **Пароли** — Argon2id (m=19456, t=2, p=1), соль из crypto/rand, формат PHC, сравнение в постоянном времени.
- **Защита от перебора** — блокировка учётной записи на 30 минут после 10 неудач + per-IP лимит на /login (10 за 5 минут).
- **Аудит входов** — таблица auth_events фиксирует успешные/неудачные входы, блокировки, refresh, выход, повторное использование токена (с IP/UA).
- **Refresh-cookie** — HttpOnly, Secure (в production), SameSite=Strict; в JSON refresh-токен не отдаётся.

8.2. Авторизация (RBAC)

- **Модель** — ролевая (RBAC) на базе PermissionResolver: объединение прав ролей, групп пользователя и точечных переопределений.
- **Приоритеты** — бан -> запрет (высший приоритет); супер-админ -> полный доступ; при конфликте deny побеждает allow.
- **Гранулярные ключи** — page.admin, page.admin.blacklist, permission.audit.read/manage, action.ban.user и др. Кэш прав in-memory с TTL 30 с.
- **Журнал отказов** — каждый ответ 403 фиксируется в таблице access_denials (IP, UA, ресурс, причина).

Область применения RBAC: Гранулярная проверка прав реально применена к части операций (ЧС, аудит, баны, документы). Часть protected-маршрутов защищена только проверкой JWT. При описании доступа корректно говорить «гранулярный доступ к критичным операциям», а не «ко всем маршрутам».

8.3. Защита от типовых атак

Угроза	Мера защиты
SQL-инъекции	GORM + параметризованные запросы (плейсхолдеры), ORDER BY не из пользовательского ввода
XSS	Санитизация v-html через DOMPurify (whitelist) в frontend/src/utls/sanitize.js
CSRF	Access — через Authorization: Bearer (не cookie); refresh — SameSite=Strict HttpOnly

	cookie
Brute-force	Rate limiting + login-лимит + блокировка учётной записи; Argon2id растягивает каждую попытку
Перебор частотой / DoS	Глобальный rate limiter (200/60 с) + login-limiter (10/5 мин)
Некорректный ввод	go-playground/validator, enum-whitelist, лимит размера и MIME загрузок
Паники сервера	Recover-middleware, generic-ошибки клиенту, детали — только в лог

8.4. Сеть и инфраструктура

- **Security-заголовки** — на двух уровнях: Echo (X-Frame-Options, nosniff, CSP default-src self, HSTS) и nginx (HSTS preload, X-Frame, nosniff, Referrer-Policy, Permissions-Policy, CSP).
- **HTTPS** — TLS терминируется на внешнем уровне (VPS), внутренний обмен — по HTTP внутри сети Docker. Staging под Basic Auth.
- **Docker** — multi-stage сборка, образ на alpine, non-root пользователь (uid 1001), статический бинарь, HEALTHCHECK; CI проверяет запуск от non-root.
- **Секреты** — только через env; JWT-секреты и DATABASE_URL обязательны и валидируются при старте; .env не хранится в репозитории.

9. Соответствие 152-ФЗ и защита ПДн

Система реализует технические меры защиты персональных данных: шифрование на уровне полей, аудит доступа и журнал согласий.

9.1. Шифрование персональных данных

- **Алгоритм** — AES-256-GCM (аутентифицированное шифрование) для данных at rest.
- **Что шифруется** — серия и номер паспорта (passport_series_number), номер патента (patent_number) на моделях Employee, UniqueEmployee, ApplicationEmployee.
- **Прозрачность** — шифрование/расшифровка выполняются GORM-хуками BeforeSave/AfterFind, бизнес-логика их не видит.
- **Поиск по зашифрованным полям** — детерминированный HMAC-SHA256 в отдельных колонках *_hmac с индексами (WHERE по HMAC без расшифровки таблицы).

9.2. Аудит доступа к ПДн

- **Механизм** — middleware PDAudit логирует обращения к эндпоинтам с ПДн.
- **Покрытие** — /employees, /unique-employees, /attachments.
- **Состав записи** — пользователь, действие (по HTTP-методу), ресурс, IP, метод, путь, статус-код, время.
- **Хранение** — таблица pd_audit_logs, запись асинхронная (в горутине с логированием ошибок).

9.3. Журнал согласий

- **API** — /consents: выдать (POST), отозвать (DELETE /:type), список (GET), проверить активность (GET /check/:type).
- **Модель** — PDConsent: user_id, тип согласия (pd_processing / pd_transfer), статус, IP, UA, метки времени выдачи и отзыва.

Ключевое условие соответствия: Шифрование ПДн ВКЛЮЧАЕТСЯ переменной DATA_ENCRYPTION_KEY. Если ключ не задан, система работает в режиме passthrough — персональные данные пишутся в открытом виде. Для соблюдения 152-ФЗ ключ обязателен в production; установка REQUIRE_ENCRYPTION=true гарантирует отказ старта без ключа.

10. CI/CD

Сборка, тестирование и доставка автоматизированы через GitHub Actions. Изменения проходят путь от пуша до автодеплойа на staging; production-деплой выполняется с подтверждением.



Рисунок 3 — Конвейер CI/CD (GitHub Actions)

10.1. Рабочие процессы (workflows)

Workflow	Назначение
ci.yml	Главный конвейер: vet/lint, тесты (go test -race + PostgreSQL, vitest), docker-build, deploy-staging (push в dev), ci-summary
e2e.yml	Playwright на PR в dev, матрица из 4 шардов, живой сервер + PostgreSQL
security.yml	govulncheck (Go) + npm audit (high+) + Trivy (CRITICAL/HIGH); push, PR и еженедельно по расписанию
site-smoke.yml	Краулер сайта: проверка доступности ключевых страниц
deploy-production.yml	Деплой на production по push в prod или ручному запуску с подтверждением (environment approval)

10.2. Сканирование безопасности

- govulncheck** — поиск известных уязвимостей в Go-зависимостях.

- **npm audit** — аудит frontend-зависимостей (high и выше для prod-зависимостей — блокирует).
- **Trivy** — сканирование Docker-образов (уровни CRITICAL/HIGH, ignore-unfixed).
- **dependency-review** — проверка зависимостей в PR.

Ветки и деплой: dev — интеграционная (автодеплой на staging); staging — синхронизируется с dev; prod — production (автодеплой с approval). Feature-ветки issue/N сливаются в dev через PR и удаляются после merge.

11. Сопровождение и эксплуатация

11.1. Резервное копирование PostgreSQL

Бэкап базы средствами pg_dump из контейнера БД:

```
docker compose -f docker-compose.prod.yml exec db \
  pg_dump -U postgres auto_registry > backup.sql
```

Восстановление из дампа:

```
docker compose -f docker-compose.prod.yml exec -i db \
  psql -U postgres auto_registry < backup.sql
```

Рекомендация: Регулярное резервное копирование настраивается планировщиком (cron) с хранением копий вне хоста БД. Персистентность данных обеспечивается named volume postgres-data.

11.2. Мониторинг и проверка состояния

Статус сервисов:

```
docker compose -f docker-compose.prod.yml ps
```

Просмотр логов:

```
make deploy-logs
```

Проверка работоспособности (health-check):

```
curl http://localhost/health # {"status":"ok"}
```

О границах health-check: Эндпоинт /health подтверждает, что процесс жив, но не гарантирует исправность всего API. Для полноценной проверки после деплоя дополнительно дёргают реальный прикладной эндпоинт. У контейнеров настроен HEALTHCHECK.

11.3. Обновление

Шаг 1. Получить изменения

Подтянуть новую версию кода в каталог развёртывания (git pull / fetch нужной ветки).

Шаг 2. Пересобрать образы

Выполнить сборку production-образов.

```
make deploy-build
```


Шаг 3. Перезапустить сервисы

Поднять обновлённые контейнеры; миграции схемы применятся автоматически при старте backend.

```
make deploy-up
```

Шаг 4. Проверить

Убедиться в работоспособности (раздел 11.2) и проверить ключевой прикладной сценарий.

11.4. Откат

- **Откат кода** — через git revert проблемного коммита и повторный деплой (для production — revert merge-коммита dev -> prod).
- **Диагностика упавшего деплоя** — SSH на VPS, просмотр логов backend контейнера (docker compose ... logs backend --tail=200).

Откат схемы БД: AutoMigrate не откатывает изменения схемы. При откате версии, изменявшей структуру таблиц, учитывайте необратимость и опирайтесь на резервную копию (раздел 11.1).

12. Архитектурные решения (ADR)

Ключевые архитектурные решения зафиксированы в формате ADR (контекст, решение, последствия). Перечень:

№	Решение	Статус
001	Go вместо Rust (Echo v4, GORM)	Принято
002	Echo v4 как HTTP-фреймворк	Принято
003	GORM с AutoMigrate	Принято, требует пересмотра для production
004	AES-256-GCM + HMAC-SHA256 для шифрования ПДн	Принято (152-ФЗ)
005	Pinia вместо Vuex	Принято
006	Options API вместо Composition API	Принято
007	Кастомные skeleton-компоненты	Принято
008	In-memory rate limiter	Принято, с ограничениями
009	pgAdmin вместо Adminer (dev only)	Принято

Кратко об отдельных решениях:

- **ADR-001 (Go вместо Rust)** — переход на Go ускорил разработку и упростил порог входа; совместимость со схемой БД сохранена через детекцию существующих таблиц в AutoMigrate.
- **ADR-004 (AES-GCM + HMAC)** — шифрование ПДн через GORM-хуки прозрачно для бизнес-логики; HMAC обеспечивает поиск без расшифровки. Ограничения: нет ротации ключей, base64-overhead.
- **ADR-008 (in-memory rate limiter)** — нет внешних зависимостей и сетевых задержек; не подходит для горизонтального масштабирования (счётчики у каждого инстанса свои, сбрасываются при рестарте) — при росте заменить на внешнее хранилище.

Где смотреть полностью: Полные тексты ADR с разделами «Контекст / Решение / Последствия» — в каталоге docs/adr/ репозитория.

Лист регистрации изменений

В таблице фиксируются изменения документа по мере развития системы.

Дата	Версия	Описание изменения	Автор
20.06.2026	1.0	Первая редакция технической документации	Бюро пропусков